ASSIGNMENT – 10.2

Name: Yuvaraj.B

H.No: 2303A51440

Batch: 21

- **Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability**
  Lab Objectives

  • Use AI for automated code review and quality enhancement.
  • Identify and fix syntax, logical, performance, and security issues in Python code.
  • Improve readability and maintainability through structured
  Week5 -
  Tuesday

- refactoring and comments.
  • Apply prompt engineering for targeted improvements.
  • Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices
  Task Description -1(Error Detection and Correction) Task:
  Use AI to analyze a Python script and correct all syntax and logical errors.

  Sample Input Code:

  ```
  def calculate_total(nums)
  sum = 0

  for n in nums
  sum += n
  return total
  ```

  Expected Output-1:

  Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Prompt:

“Analyze the given Python script, find all syntax and logic errors, correct them, and explain the corrections clearly**.”**

CODE:

ai123.py > ...

```python
'''Analyze the given Python script, find all syntax and logic errors, correct them, and explain the corrections clearly
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total'''
def calculate_total(nums):
    """Calculate the total sum of a list of numbers.

    Args:
        nums (list): A list of numbers to be summed.

    Returns:
        int: The total sum of the numbers in the list.
    """
    total = 0  # Initialize total to 0
    for n in nums:  # Iterate through each number in the list
        total += n  # Add the current number to the total
    return total  # Return the final total
# Example usage:
numbers = [1, 2, 3, 4, 5]
print(calculate_total(numbers))  # Output: 15
```

PROBLEMS 135 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c
rs/2303A/OneDrive/Desktop/python course/ai123.py"
15
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

1. AI detected both syntax and logical errors accurately.
2. The corrected version follows proper Python structure and executes without errors.
3. The variable naming issue was resolved, preventing future bugs.
4. Code quality improved significantly through error correction and proper indentation.

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b
print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

Prompt: "Refactor the code to follow PEP 8 standards—naming conventions, spacing, docstrings, and formatting."

CODE:

```python
'''Refactor the code to follow PEP 8 standards—naming conventions, spacing, docstrings, and formatting
def findSum(a,b):return a+b print(findSum(5,10)) '''
def find_sum(a, b):
    """Returns the sum of two numbers.

    Args:
        a (int): The first number.
        b (int): The second number.

    Returns:
        int: The sum of a and b.
    """
    return a + b
print(find_sum(5, 10))
```

```
PROBLEMS 132   OUTPUT   DEBUG CONSOLE   TERMINAL 9   PORTS

PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai123.py"
15
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

1. AI applied PEP 8 rules correctly, making the code more standardized.
2. Readability improved through consistent spacing and proper naming.
3. The use of a docstring increases clarity and maintainability.
4. The final output matches industry-standard formatting.

Task Description -3(Code Clarity Improvement) Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):
return x-y*2
print(f(10,3))
```

Expected Output-4:

Python code rewritten with meaningful function and variable names,

proper indentation, and improved clarity.

Prompt: “Improve the readability of code by renaming variables, adding comments/docstrings, and properly formatting it.”

ai123.py > ...

```python
'''Improve the readability of code by renaming variables, adding comments/docstrings, and properly formatting it.
def f(x,y):
return x-y*2
print(f(10,3)) '''
def calculate_difference(x, y):
    """Calculate the difference between x and twice y.

    Args:
        x (int): The first number.
        y (int): The second number to be multiplied by 2 and subtracted from x.

    Returns:
        int: The result of x minus twice y.
    """
    return x - y * 2
# Example usage
if __name__ == "__main__":
    result = calculate_difference(10, 3)
    print(result)  # Output: 4
```

PROBLEMS 135 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai123.py"
4
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

1. AI improved readability without changing functionality.
2. Meaningful names make the logic easier to understand.
3. Added documentation improves maintainability.
4. Code looks more professional and beginner-friendly.

Task Description-4(Structural Refactoring) Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
print("Hello Sita")
print("Hello Ravi")
```

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

Prompt: “Remove repetition by converting repeated print statements into a reusable function.”

CODE:

ai123.py > ...

```python
'''Remove repetition by converting repeated print statements into a reusable function.
print("Hello Ram")
print("Hello Sita")
print("Hello Ravi") '''
def greet(name):
    print(f"Hello {name}")
greet("Ram")
greet("Sita")
greet("Ravi")
```

PROBLEMS 131 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Sc
rs/2303A/OneDrive/Desktop/python course/ai123.py"
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

1. AI effectively converted repetitive code into modular functions.
2. The new structure supports adding more names easily.
3. Improves maintainability and reduces redundancy.
4. 0Code follows the DRY principle (Don’t Repeat Yourself).

Task Description-5(Efficiency Enhancement) Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Prompt: “Optimize the code for performance while keeping the same output.”

```
'''Optimize the code for performance while keeping the same output.
numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers)) '''
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```

```
PRO  Focus folder in explorer (ctrl + click)  LE  TERMINAL 9  PORTS
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python cours
rs/2303A/OneDrive/Desktop/python course/.venv/ai2.py"
499999
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

1. AI optimized performance by removing unnecessary loop overhead.
2. List comprehension improved speed and reduced code size.
3. Final code is more Pythonic and efficient.

Demonstrates AI's ability to enhance algorithmic efficiency